

CS 112

Introduction to Data Structures

WEEK 13

STL set and map

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 You've earned the STL
- 2 `set` — unique and sorted
- 3 Iterating a container
- 4 `map` — keys to values
- 5 The `[]` trap **KEY**
- 6 Choosing a container
- 7 Ordered vs. unordered

You Built These by Hand

Dynamic array, linked list, stack, queue, binary search tree, self-balancing tree. The STL ships all of them.

Knowing what's inside is the point of the last twelve weeks. From here on, use the library — but now you know *why* set is fast, and what it costs.

set: Unique, Sorted, Fast

```
#include <set>

set<string> enrolled;

enrolled.insert("ada");
enrolled.insert("grace");
enrolled.insert("ada");           // ignored – already there

cout << enrolled.size();         // 2
cout << enrolled.count("ada");   // 1  (0 or 1, never more)
```

Two guarantees: **no duplicates**, and iteration comes out **in sorted order**. Both fall out of the balanced tree underneath.

What's Underneath

Operation	Complexity	Why
insert	$O(\lg n)$	walk down the tree, rebalance
count / find	$O(\lg n)$	the guessing game again
erase	$O(\lg n)$	remove, then rebalance
iteration	$O(n)$ total	in-order traversal

A balanced BST with a friendly interface. Every cost on this slide is one you derived yourself in weeks 11 and 12.

Iterating

```
for (const string &name : enrolled) {           // sorted order
    cout << name << endl;
}

set<string>::iterator it = enrolled.find("ada");
if (it != enrolled.end()) {
    cout << *it << " is enrolled" << endl;
}
```

find returns end() when there's no match — comparing against end() is how you ask "was it there?". Never dereference end(); it points past the last element, not at one.

What does this print?

```
set<int> s;  
s.insert(7);  
s.insert(3);  
s.insert(7);  
s.insert(9);  
cout << s.size();
```

- A. 3
- B. 4
- C. 2
- D. It won't compile

map: Keys to Values

```
#include <map>

map<string, int> scores;

scores["ada"]    = 95;
scores["grace"] = 98;

cout << scores["ada"];           // 95
cout << scores.size();           // 2
```

Python's dictionary, with two differences: the keys are kept **sorted**, and lookup is $O(\lg n)$ rather than average $O(1)$.

Walking a map

```
for (const auto &entry : scores) {  
    cout << entry.first << ": " << entry.second << endl;  
}  
  
// ada: 95  
// grace: 98      ← always in key order
```

Each element is a pair: `.first` is the key, `.second` is the value. The keys come out sorted because a map is the same balanced tree, keyed on `.first`.

What does `scores.size()` report?

```
map<string, int> scores;  
scores["ada"] = 95;  
  
if (scores["bob"] > 90) {      // just a lookup?  
    cout << "bob did well";  
}  
cout << scores.size();
```

- A. 1
- B. 2
- C. 0
- D. It throws an exception

The [] Trap

Inserts if absent

```
m["missing"]    // creates it!
```

Asks without inserting

```
m.count("missing")  
m.find("missing") != m.end()
```

This bites everyone once: a loop that only *reads* a map quietly doubles its size. The behaviour is deliberate — `m[k]++` as a word counter depends on it — but it means `[ ]` is not a read-only operation.

A Word Counter

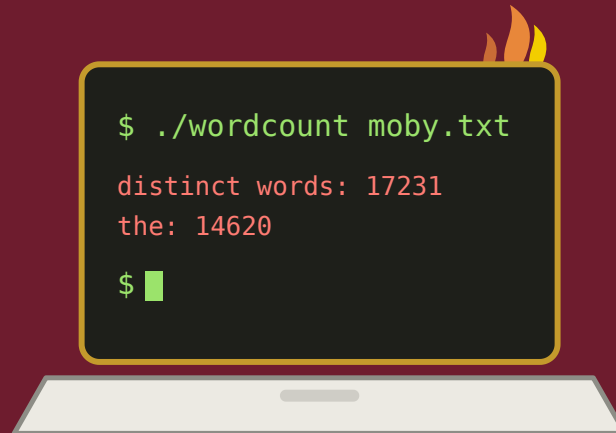
```
map<string, int> counts;
string word;

while (fin >> word) {
    ++counts[word];           // creates it at 0, then increments
}

for (const auto &entry : counts) {
    cout << entry.first << " " << entry.second << endl;
}
```

Eight lines, alphabetised output, $O(\lg n)$ per word. Try writing that with an array of structs and you'll appreciate the container.

Demo



Counting words in a book with eight lines of code.

(and finding out how often 'the' appears)

Ordered or Unordered?

	<code>set</code> / <code>map</code>	<code>unordered_set</code> / <code>unordered_map</code>
Underneath	balanced tree	hash table
find / insert	$O(\lg n)$	$O(1)$ average, $O(n)$ worst
Iteration order	sorted	unspecified
Needs	<code><</code> on the key	a hash function

If you need sorted output or range queries, take the tree. If you only ever ask "is it there?", the hash table is faster — and that's next week.

Choosing a Container

- Index by position, mostly appending → `vector`
- Constant insert/remove at the ends → `deque`, `stack`, `queue`
- Membership, no duplicates, sorted → `set`
- Key to value, sorted keys → `map`
- Membership or lookup as fast as possible, order irrelevant → `unordered_*`

The question is never "which container is best?" — it's "which operations will I do a million times?"

Week 13 Recap

- `set` and `map` are balanced trees with a friendly interface
- Both keep keys **sorted**; both are $O(\lg n)$ per operation
- A set holds each value once; a map holds each key once
- `m[k]` **inserts** when the key is missing — use `count` to ask
- Need order? tree. Need raw speed? hash table

Next: a22 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

